

Rapport de TP

Reconnaissance de formes – Application à la classification de feuilles

1. Introduction

L'objectif de ce travail pratique (TP) a été de développer une application capable de reconnaître des formes à partir d'images. J'ai choisi le thème de la **reconnaissance de feuilles d'arbres** (exemples : chêne, érable, bouleau, saule).

Ce projet illustre les différentes étapes d'un processus de vision par ordinateur :

1. Prétraitement des images,
2. Extraction automatique de caractéristiques géométriques,
3. Constitution d'un jeu de données,
4. Entraînement d'un modèle de classification,
5. Évaluation et test sur de nouvelles images.

L'application a été développée en Python à l'aide des bibliothèques OpenCV, NumPy, Pandas, Scikit-learn et Matplotlib.

2. Description de l'application

2.1 Type d'application

Application de reconnaissance de formes naturelles, centrée sur la classification de feuilles d'arbres selon leur morphologie.

2.2 Type d'images traitées

- Images couleur (format JPG)
- Chaque image contient une seule feuille sur fond sombre
- Dimensions variables (automatiquement redimensionnées et seuillées)

2.3 Données utilisées

Les images proviennent du jeu de données “*Leaf Classification*” disponible sur [Kaggle](#).

Elles ont été triées manuellement en quatre classes distinctes :

- chene / (chêne)
- erable / (érable)
- bouleau / (bouleau)
- saule / (saule)

Chaque classe contient environ 15 images, soit 60 images au total.

3. Méthodologie

3.1 Prétraitement des images

Les étapes de prétraitement ont été réalisées avec la bibliothèque OpenCV :

1. Conversion en niveau de gris
2. Floutage gaussien pour réduire le bruit
3. Seuillage adaptatif (méthode de Gauss) pour séparer la feuille du fond
4. Extraction du contour principal

Ces étapes permettent d’obtenir une image binaire propre à partir de laquelle les caractéristiques géométriques peuvent être calculées.

3.2 Caractéristiques extraites

Pour chaque feuille, plusieurs paramètres géométriques et de forme ont été calculés :

Caractéristique	Description	Fonction OpenCV
Aire	Surface de la feuille	cv2.contourArea()
Périmètre	Longueur du contour	cv2.arcLength()
Rapport d’aspect	Largeur / hauteur du rectangle englobant	cv2.boundingRect()
Extent	Aire / aire du rectangle englobant	cv2.boundingRect()
Solidity	Aire / aire de l’enveloppe convexe	cv2.convexHull()

Compacité	$(4\pi \times \text{Aire}) / \text{Périmètre}^2$	Calcul manuel
Moments de Hu (7)	Descripteurs invariants de forme	cv2.HuMoments()

Chaque image est donc représentée par un vecteur de 13 caractéristiques numériques.

3.3 Constitution du jeu de données

Un script Python parcourt automatiquement le dossier *dataset/*, extrait les caractéristiques de chaque image et les enregistre dans un fichier CSV :

feuilles_features.csv.

4. Méthode de reconnaissance

4.1 Choix du modèle

Le modèle de classification choisi est le **Random Forest Classifieur** (méthode d'ensemble basée sur des arbres de décision). Il est robuste, rapide à entraîner et bien adapté aux petits jeux de données comportant des variables numériques variées.

4.2 Entraînement

- Jeu d'entraînement : 80 % des données
- Jeu de test : 20 % des données
- Normalisation : Standardisation (z-score) via *StandardScaler()*

Conclusion

Ce TP a permis de construire une chaîne complète de **reconnaissance de formes** basée sur des techniques classiques de vision par ordinateur. Le modèle développé, fondé sur l'extraction de caractéristiques géométriques et le classifieur Random Forest, a atteint **une précision de 83.33 %** sur le jeu de test interne.

L'application démontre :

- la pertinence des descripteurs de forme (aire, compacité, moments de Hu),
- la puissance des méthodes classiques sans réseaux de neurones,
- et la faisabilité d'une reconnaissance fiable avec peu de données.

Améliorations possibles :

1. Ajouter plus d'images par classe avec des variations (rotation, luminosité, angles)
2. Enrichir les descripteurs de forme (texture, couleur moyenne)
3. Utiliser une validation croisée pour mieux évaluer la robustesse
4. Comparer avec des modèles plus complexes (SVM, Gradient Boosting)